

The Long Game — Handover Guide

For the person taking over this project. You don't need to know how to code. You need VS Code, GitHub Copilot, a web browser, and this guide. Everything below is written for a non-programmer — where things live, how to change them safely, how to test, and how to publish.

1 · What this is, in 30 seconds

"The Long Game" is a set of Capital Group-branded investing games. A player gets \$50,000, faces real historical market moments without being told the dates, chooses Buy / Hold / Sell, and at the end sees the big reveal: every moment was real, and simply staying invested usually won.

Each game is **one single .html file** — the words, the design, the data, and the logic are all inside that one file. Open it in a browser and it runs. There is no installation, no database, no build step. This is the most important fact in this whole document: **one file = one complete game.**

2 · The files — what's what

File	What it is	Do you edit it?
<code>long-term-game-live.html</code>	Live Room — the audience event version. Big screen + phones voting via QR code.	Yes
<code>long-term-game-learning.html</code>	Learning — the solo, product-agnostic version. No fund mentioned.	Yes
<code>long-term-game-sales.html</code>	Sales — the solo version WITH the American Balanced Fund panel and fund line on the chart.	Yes

File	What it is	Do you edit it?
<code>long-term-investing-game.html</code> , <code>long-term-investing-game-2020.html</code>	Older prototypes, kept for reference only.	No — leave them alone
<code>VERIFICATION-CHECKLIST.md</code>	The list of every number that must be verified and every compliance sign-off needed before real client use .	Read it. Update it when data changes.
<code>live-server/</code> folder	The kit for running the Live Room on a company laptop with no internet services. Has its own plain-English <code>README.md</code> .	Only if events need it
<code>HANDOVER.md</code>	This document.	Keep it current

The public demo site (while it's up) is at:

<https://cheesin30.github.io/decisionframework/> — the Live Room is the homepage; the other games are at `/long-term-game-learning.html` and `/long-term-game-sales.html`.

3 · The golden rules

1. **Small changes, tested immediately.** Change one thing, save, refresh the browser, play it. Never batch up ten edits and hope.
2. **Copilot is your hands, the browser is your judge.** Whatever Copilot writes, YOU confirm it works by actually playing the game.
3. **Never edit below the "vendored" line.** Near the top of each game file there are two giant walls of unreadable compressed code labeled `Chart.js` and `html2canvas` (they draw the charts and make the share image). They are someone else's library, pasted in so the game works offline. Scroll past them; never touch them.
4. **Don't weaken the legal text.** Every screen has disclaimers ("Illustrative... past performance..."). Compliance depends on them. You can fix a typo; you cannot delete or soften them without a compliance conversation.

5. **When in doubt, undo.** `Ctrl+Z` undoes typing. The Source Control panel can discard everything back to the last saved version (Section 9). You cannot permanently break anything that has been committed to GitHub.
-

4 • Your daily setup

1. Open **VS Code** → `File` → `Open Folder...` → pick the `decisionframework` folder (clone it from GitHub first if you haven't: in VS Code press `Ctrl+Shift+P`, type "Git: Clone", paste the repo address).
2. Click a game file in the left sidebar to open it.
3. To **see the game**: find the file in Windows Explorer / Finder and double-click it — it opens in your browser. Keep that tab open.
4. Your loop is: **edit in VS Code** → `Ctrl+S` to save → **refresh the browser tab** (`F5`) → **play**.

The one navigation skill you need: `Ctrl+F` (find). These files are long, but every section has a searchable name. All the "where is..." answers in Section 6 are search terms.

5 • How to make changes with Copilot (the workflow)

You will rarely write code. You will *describe* changes and let Copilot make them. Three habits:

A. Ask before you touch. Select a chunk of code, open Copilot Chat, and ask: *"Explain what this does in plain English."* Do this until you're confident you're in the right place.

B. Small, specific instructions. Select the relevant lines, press `Ctrl+I` (inline Copilot), and describe the change like you'd brief a colleague:

"Change this headline to 'Could you have held your nerve?' — keep all the HTML tags exactly as they are."

"In this scenario object, rewrite the 'story' text to mention rising oil prices. Don't change any of the field names or the month."

"Make this button's background the brand navy (--navy) instead of teal."

C. Tell Copilot the house rules. For anything bigger than a wording change, start your prompt with:

"This is a single self-contained HTML file. Don't add any external libraries, links to the internet, or new files. Keep the existing style. Make the smallest change that does this: ..."

That one sentence prevents 90% of vibe-coding accidents (Copilot loves to add internet-loaded libraries, which would break the game on locked-down networks — the whole point of these files is that they work offline).

After every Copilot change: save, refresh, play through. If the screen goes blank or a button dies, don't debug — undo (**Ctrl+Z** until it's back), then try again with a smaller, clearer instruction.

6 · Where everything lives (Ctrl+F cheat sheet)

These search terms work in each game file:

You want to change...	Search for...	Notes
Welcome screen / headlines / button labels	Just Ctrl+F the exact words you see on screen	All visible text is plain text in the file
The 12 game scenarios (stories the player faces)	const P00L	See recipe 7.2 before editing
The historical era stories (1973, 1987, 2000, 2008...)	const HIST_ERAS	Titles, takeaways, "about" text per era
Starting money (\$50,000)	const START	See recipe 7.3 — also update the on-screen wording
Brand colors	:root	Navy --navy , teal --accent , green, etc. Change here, whole game follows
Fund facts panel (Sales only): ticker,	fund-card	Plain HTML — numbers are just text

You want to change...	Search for...	Notes
expense ratio, returns table		
Fund line on the chart on/off (Sales only)	FUND_LINE_ENABLED	true = shown, false = hidden
The fund's chart data (Sales only)	AMBAL_NET_2020	⚠ Read recipe 7.5 — this is approximated data with rules attached
Market data (the real monthly returns)	EQ_2020, AGG_2020, BENCH_0525	⚠ Don't edit unless replacing with verified data — update VERIFICATION- CHECKLIST.md if you do
Live Room's online service address	const LIVE_BACKEND	⚠ Recipe 7.7 — the line's exact shape matters
Legal / disclaimer text	disclaimer or Illustrative	Rule 4 applies

Every file also starts with a long comment block (the text between `<!--` and `-->` at the very top) explaining that game's design — worth reading once.

7 · Recipe cards

7.1 Change any wording the player sees

`Ctrl+F` the exact sentence → edit it right there (or select it and tell Copilot the new wording) → save → refresh → check that screen.

7.2 Edit or add a game scenario

Search `const POOL`. Each scenario looks like:

```
{ title: "The world stops", month: "2020-03", emotionalPull: "panic + fear",
  story: "...what the player reads during play...",
```

```
mood: "...one-line market feel...",  
  
event: "The COVID-19 crash · March 2020",  
  
what: "...the reveal explanation..." },
```

- **Safe to edit freely:** `title`, `story`, `mood`, `emotionalPull`, `event`, `what` — that's all wording.
- **Handle with care:** `month`. The game looks up the REAL market return for that month — nothing about the market move is hand-typed. If you add a new scenario, pick a genuinely dramatic real month between `2020-01` and `2025-12` and the numbers take care of themselves.
- The game randomly draws 6 of the 12 each play, always shown in date order. Keep the pool at a comfortable size (10–14) so replays feel fresh.
- Good Copilot prompt: *"Add one more scenario object to POOL about [real event] in [YYYY-MM], written in the same second-person emotional style as the others. Don't change anything else."*

7.3 Change the starting amount

Search `const START` → change `50000`. Then `Ctrl+F 50,000` and `$50k` through the file — the amount is also written into on-screen sentences, and those don't update themselves. (Ask Copilot: *"Find every place this file says \$50,000 in visible text and change it to \$100,000."*)

7.4 Annual refresh of the fund panel (Sales)

Once a year (or when marketing asks): search `fund-card`, update the returns table, the "as of" date, and the expense ratio from the fund's **current prospectus/fact sheet** (not a random website). Then update `VERIFICATION-CHECKLIST.md` §1.1 so the paper trail stays honest.

7.5 The fund line on the chart (Sales) — read before touching

The purple dashed fund line is currently drawn from an **approximation**: each calendar year lands exactly on the fund's published annual return, but the month-to-month wiggle follows the benchmark. That's disclosed on screen and in `VERIFICATION-CHECKLIST.md` §1.1. The rules: - If compliance/data ever supplies the **real verified monthly series**, replace the `AMBAL_NET_2020` array with it (Copilot prompt: *"Replace the values in AMBAL_NET_2020 with this list, keeping the same format: ..."*), and then delete/soften the "approximated" wording in the fund note and bottom disclaimer — search `approximat` to find all of it. - To hide the line entirely (e.g. compliance says

pause), set `FUND_LINE_ENABLED = false`. The note under the chart auto-switches to a "pending verification" message. That's the whole off-switch. - Never "improve" the fund's numbers by hand. Every figure must trace to a published source.

7.6 Change the brand colors

Search `:root` (near the top, in the style section). You'll see the palette: `--navy: #083469`, `--accent: #008074` (teal), etc. Change the color codes here and the whole game follows. Test afterwards — especially text readability on the dark reveal cards.

7.7 Live Room: the service address

Search `const LIVE_BACKEND` in `long-term-game-live.html`. It looks like:

```
const LIVE_BACKEND = { databaseURL: "https://...firebase.database.app" };
```

- That address is the Firebase "switchboard" that passes votes from phones to the big screen. Replace the address only if you set up a new Firebase Realtime Database (or an IT-hosted relay).
 - **Do not reformat this line.** The company-laptop servers in `live-server/` find this exact line and auto-rewrite it when they serve the page. Change only the address between the quotes; leave the shape of the line alone.
 - Everything about running events (company laptops, hotspots, what to ask IT) is in `live-server/README.md` — written for non-technical readers.
-

8 · Test before you publish (10 minutes)

After ANY change, play the changed game start to finish. Before publishing, check these specifically — they're the invariants that have caught real bugs:

Solo games (Learning & Sales): - [] Portfolio starts at exactly **\$50,000** at the first moment. - [] Play a round pressing **Hold every time** → at the reveal, "Your choices" and "Stayed invested" must be **identical to the dollar**. If they differ, something broke — undo your change. - [] Click every era tab (1973, 1987, 2000, 2008, 2011, 2015–18) and back to "Your 2020" — the whole section should swap cleanly each way. - [] "Play again" lets you pick a different risk profile. - [] Sales only: fund line appears on the 60/40 profile, disappears on era views, and the 70/30 profile shows the "reselect Balanced" note. - [] Make the browser window phone-narrow (or press `F12` → phone icon) — nothing should overflow sideways.

Live Room: - [] Open `?live=host` in one window → room code appears. - [] Join from a second window (or your phone) with that code → vote → the phone should NOT show the outcome until the host presses "**Reveal what happened**". - [] Leaderboard appears at the end.

9 · Saving and publishing (GitHub, in plain English)

Think of it as two shelves: - **main branch** = the master copy of the files. - **gh-pages branch** = the live public website. Whatever sits on this shelf IS what the world sees at the site address.

Saving your work (the master copy). In VS Code, click the **Source Control** icon (left bar, branching-lines symbol). You'll see your changed files. Hover a file to see the diff (what you changed). Then: 1. Click **+** next to the file (this "stages" it — puts it in the box). 2. Type a one-line description of what you changed. 3. Click **Commit**, then **Sync Changes** (pushes it to GitHub).

That's it. Every commit is a permanent restore point — this is your backup system, which is why you commit often.

Publishing to the live site. The simplest reliable way is the GitHub website: 1. Go to the repo on github.com and switch the branch dropdown (top-left of the file list) to **gh-pages**. 2. Click **Add file → Upload files**, drag in the game file(s) you changed, and commit. Uploading a file with the same name replaces the old one. 3. Wait ~1 minute, then hard-refresh the public page (**Ctrl+Shift+R**).

Note: on **gh-pages** the Live Room file is ALSO saved as `index.html` (that's why it's the homepage). If you changed `long-term-game-live.html`, upload it twice — once as itself, and once renamed to `index.html`.

Undo, at every level: - Typing mistake → **Ctrl+Z**. - "I want to throw away everything since my last commit" → Source Control → right-click the file → **Discard Changes**. - "I published something bad" → on github.com, open the file's **History**, view the last good version, and re-upload that. (Or ask Copilot Chat: *"Show me the git commands to restore this file to the previous commit."*)

10 · The rules that outrank everything (compliance)

This is a Capital Group-branded prototype with real market data and real fund

performance in it. It is currently **internal / pre-approval**:

1. **VERIFICATION-CHECKLIST.md is the contract.** Nothing goes in front of a real client or external audience until the data verifications and the compliance/brand sign-offs in that file are done. If you change any number, record it there.
 2. **The public demo site is temporary.** The recorded decision: it stays up for testing, and **must be taken down before the compliance review concludes** — delete the `gh-pages` branch on GitHub AND lock the Firebase database rules. Don't inherit this project and forget that.
 3. **Disclaimers stay.** (Rule 4 again, because it matters.)
 4. **The Live Room collects nothing but nicknames and votes**, in throwaway rooms. Keep it that way — the moment someone suggests collecting emails through it, that's a new privacy conversation, not a Copilot prompt.
-

11 · When something breaks and undo isn't enough

- **Blank page after an edit:** almost always a broken piece of JavaScript. Press `F12` in the browser → **Console** tab → screenshot the red error → paste it to Copilot Chat with *"This single-file HTML game shows a blank page, here's the console error, here's the code near it — what's the smallest fix?"*
 - **Chart looks wrong / numbers seem off:** re-run the Section 8 checklist, especially the all-Hold tie. If the tie fails, your change touched game math — discard the change and re-approach with a narrower prompt.
 - **Live Room won't connect:** it now fails loudly with an on-screen message after ~8 seconds instead of hanging. The message itself lists the likely causes (corporate network blocking, etc.). `live-server/README.md` has the full plain-English troubleshooting list, including the phone-hotspot fallback for events.
 - **Totally lost:** every version of every file ever committed is on GitHub under the repo's commit history. You can always get back to a working game.
-

12 · Tiny glossary

Word	Meaning here
Repo	The project folder as it exists on GitHub

Word	Meaning here
Branch	A named shelf of the files (<code>main</code> = master copy, <code>gh-pages</code> = live site)
Commit	A saved snapshot with a description — your restore points
Push / Sync	Send your commits up to GitHub
HTML / CSS / JavaScript	The words / the styling / the behavior — all inside the one file
Firebase	Google's small online service the Live Room uses to pass votes around
<code>const SOMETHING =</code> <code>...</code>	"Here is a named value the game uses" — the settings knobs live in lines like this